

BEGINNER → ADVANCED

SYSTEMS PROGRAMMING

# Rust — The Complete Guide

Ownership · Borrowing · Traits · Concurrency · Every core topic with theory and code

---

Prepared study companion

A hands-on reference from first program to advanced patterns

# Contents

- 1. Introduction, Installation & Cargo**
- 2. Variables, Mutability & Constants**
- 3. Data Types**
- 4. Functions & Control Flow**
- 5. Ownership**
- 6. References & Borrowing**
- 7. Slices**
- 8. Structs**
- 9. Enums & Pattern Matching**
- 10. Collections (Vec, String, HashMap)**
- 11. Error Handling (Result & Option)**
- 12. Generics**
- 13. Traits**
- 14. Lifetimes**
- 15. Closures & Iterators**
- 16. Smart Pointers**
- 17. Concurrency**
- 18. Modules, Crates & Testing**
- 19. Advanced Topics**

# 1. Introduction, Installation & Cargo

*Rust is a systems programming language focused on safety, speed, and concurrency. Its signature feature is **memory safety without a garbage collector**, enforced at compile time by the ownership system.*

## 1.1 Why Rust

- **Memory safety** — no null-pointer dereferences, no data races, no use-after-free, all caught at compile time.
- **Zero-cost abstractions** — high-level constructs compile down to code as fast as hand-written C.
- **Fearless concurrency** — the compiler prevents data races so multithreaded code is safe by construction.
- **No garbage collector** — deterministic performance, suitable for embedded and real-time systems.

## 1.2 Installation & first program

```
# Install via rustup (the toolchain manager)
curl --proto '=https' --tlsv1.2 https://sh.rustup.rs -sSf | sh
rustc --version      # verify the compiler
```

```
// main.rs – the classic first program
fn main() {
    println!("Hello, world!");    // println! is a macro (note the !)
}
```

```
rustc main.rs      # compile
./main             # run -> Hello, world!
```

## 1.3 Cargo — the build system & package manager

```
cargo new hello      # create a new project
cargo build          # compile (debug)
cargo build --release # optimized build
cargo run            # build + run
cargo test           # run tests
cargo check          # fast type-check without producing a binary
```

**Cargo.toml** is the manifest: it declares the package name, version, edition, and dependencies (called *crates*) pulled from crates.io.

## Viva — Rust basics

### Q What makes Rust memory-safe without a garbage collector?

A The ownership system with borrow checking enforces memory rules at compile time, so memory is freed deterministically when owners go out of scope — no runtime GC needed.

### Q What is Cargo?

A Rust's official build system and package manager — it compiles code, manages dependencies (crates), runs tests, and handles releases via the Cargo.toml manifest.

### Q Why is `println!` written with an exclamation mark?

A The `!` marks it as a macro rather than a function; macros are expanded at compile time and can take a variable number of arguments.

## 2. Variables, Mutability & Constants

```
fn main() {  
    let x = 5;           // immutable by default  
    // x = 6;           // ERROR: cannot assign twice  
    let mut y = 10;      // mut makes it mutable  
    y = 20;             // OK  
  
    const MAX: u32 = 100_000; // const: always immutable, type required  
  
    // Shadowing: reuse a name, even changing type  
    let spaces = "  ";  
    let spaces = spaces.len(); // now an integer  
    println!("{x} {y} {MAX} {spaces}");  
}
```

**Immutable by default** is a core Rust philosophy: you opt into mutation with `mut`, which makes intent explicit and prevents accidental changes.

### Viva — Variables

#### Q Are variables mutable by default in Rust?

**A** No — they're immutable by default. You must add `mut` to allow reassignment, which makes mutation explicit and safer.

#### Q Difference between `const` and `let`?

**A** `const` is always immutable, must have an explicit type, and is evaluated at compile time; `let` bindings are immutable by default but can be made mutable and are usually runtime values.

#### Q What is shadowing?

**A** Redeclaring a variable with the same name using `let`; it creates a new binding (possibly a new type) rather than mutating the original.

## 3. Data Types

Rust is statically typed — every value's type is known at compile time, though the compiler can often infer it.

### 3.1 Scalar types

| Category  | Types                                                    |
|-----------|----------------------------------------------------------|
| Integers  | i8/i16/i32/i64/i128/usize (signed), u8..usize (unsigned) |
| Floats    | f32, f64 (default)                                       |
| Boolean   | bool (true / false)                                      |
| Character | char (4 bytes, a Unicode scalar)                         |

### 3.2 Compound types

```
fn main() {  
    // Tuple: fixed-size, mixed types  
    let tup: (i32, f64, char) = (500, 6.4, 'z');  
    let (a, b, c) = tup;           // destructuring  
    println!("{}", tup.0);        // index access  
  
    // Array: fixed-size, same type  
    let arr = [1, 2, 3, 4, 5];  
    let zeros = [0; 3];           // [0, 0, 0]  
    println!("{}", arr[0]);  
}
```

#### Viva — Data types

##### Q Is Rust statically or dynamically typed?

**A** Statically typed — all types are known at compile time. Type inference lets you omit many annotations, but the type is still fixed.

##### Q Difference between an array and a tuple?

**A** An array holds multiple values of the same type with a fixed length; a tuple holds a fixed number of values of possibly different types.

##### Q What's the default integer and float type?

**A** i32 for integers and f64 for floats — chosen as good general-purpose defaults.

## 4. Functions & Control Flow

```
fn add(a: i32, b: i32) -> i32 { // typed params, -> return type
    a + b                       // last expression = return (no semicolon)
}

fn main() {
    let n = add(3, 4);

    // if is an expression
    let label = if n > 5 { "big" } else { "small" };

    // loop with a value
    let mut count = 0;
    let result = loop {
        count += 1;
        if count == 5 { break count * 2; } // loop can return a value
    };

    // while
    let mut k = 3;
    while k > 0 { k -= 1; }

    // for over a range / collection
    for i in 0..3 { println!("{i}"); } // 0,1,2
    for x in [10, 20, 30] { println!("{x}"); }

    println!("{label} {result}");
}
```

**Expressions vs statements:** a block's final expression (without a trailing semicolon) is its value. Adding a semicolon turns it into a statement that returns `()`.

### Viva — Functions & control flow

#### Q How does a Rust function return a value?

**A** Either with an explicit `return`, or implicitly by making the last line an expression without a semicolon.

#### Q Is if an expression in Rust?

**A** Yes — if/else evaluates to a value, so you can write `let x = if cond { 1 } else { 2 };`. Both branches must have the same type.

#### Q What's special about loop?

**A** It loops forever until break, and break can carry a value out, making `let v = loop { ... break x; };` possible.

## 5. Ownership

Ownership is Rust's most distinctive feature and the foundation of its memory safety. It's a set of rules the compiler checks; if any are broken, the program won't compile.

### 5.1 The three rules

1. Each value has a single **owner**.
2. There can only be **one owner at a time**.
3. When the owner goes **out of scope**, the value is dropped (freed).

### 5.2 Move semantics

```
fn main() {  
    let s1 = String::from("hello");  
    let s2 = s1;           // ownership MOVES from s1 to s2  
    // println!("{}", s1); // ERROR: s1 is no longer valid  
    println!("{}", s2);    // OK  
  
    // Clone for a deep copy if you need both  
    let s3 = s2.clone();  
    println!("{}", s2, s3); // both valid  
  
    // Copy types (integers, bool, char) are copied, not moved  
    let x = 5;  
    let y = x;           // x is still usable (stack copy)  
    println!("{}", x, y);  
}
```

**Move, not shallow copy:** assigning a heap value like `String` transfers ownership and invalidates the original to prevent double-free errors.

### 5.3 Ownership & functions

```
fn takes(s: String) { println!("{}", s); } // s dropped at end  
fn gives() -> String { String::from("hi") } // ownership moves out  
  
fn main() {  
    let a = String::from("x");  
    takes(a);           // a moved into the function; unusable after  
    let b = gives();     // b now owns the returned String  
    println!("{}", b);  
}
```



## Viva — Ownership

### Q State the three ownership rules.

A Each value has one owner; there is only one owner at a time; when the owner leaves scope, the value is dropped.

### Q What happens when you assign one String to another?

A Ownership moves — the original binding becomes invalid so both can't free the same memory (preventing double-free). Use `.clone()` for an independent deep copy.

### Q Why can you copy an integer but not a String freely?

A Simple stack-only types implement the Copy trait and are cheaply duplicated; String owns heap data, so it moves instead to keep a single owner.

### Q What is "drop"?

A When an owner goes out of scope, Rust automatically calls drop to free its resources — deterministic cleanup without a garbage collector (a form of RAI).

## 6. References & Borrowing

Borrowing lets you access a value without taking ownership, using references (`&`). The borrow checker enforces rules that guarantee no dangling references or data races.

```
fn length(s: &String) -> usize {    // borrows, doesn't take ownership
    s.len()
}                                     // s goes out of scope but doesn't drop the data

fn append(s: &mut String) {         // mutable borrow
    s.push_str(" world");
}

fn main() {
    let mut msg = String::from("hello");
    println!("len = {}", length(&msg)); // immutable borrow
    append(&mut msg);                  // mutable borrow
    println!("{}", msg);               // still owns msg -> "hello world"
}
```

### 6.1 The borrowing rules

- You may have **any number of immutable references** (`&T`), OR
- Exactly **one mutable reference** (`&mut T`) — but not both at the same time.
- References must always be valid (no dangling).

**Why the rule?** Allowing a mutable reference alongside others would let data change underneath a reader — the exact cause of data races. Rust forbids it at compile time.

### Viva — Borrowing

#### Q What is borrowing?

**A** Accessing a value through a reference (`&`) without taking ownership, so the original owner keeps the value after the borrow ends.

#### Q State the borrowing rules.

**A** At any time you can have either many immutable references or exactly one mutable reference, never both, and every reference must remain valid.

#### Q Why only one mutable reference at a time?

**A** To prevent data races — simultaneous mutation and access would make data change unpredictably. Enforcing exclusivity at compile time guarantees safety.

**Q Difference between &T and &mut T?**

**A** &T is a shared, read-only borrow; &mut T is an exclusive borrow allowing mutation.

## 7. Slices

```
fn main() {
    let s = String::from("hello world");
    let hello = &s[0..5];           // string slice: "hello"
    let world = &s[6..11];          // "world"
    let whole = &s[..];              // entire string

    let arr = [1, 2, 3, 4, 5];
    let mid = &arr[1..4];           // slice of an array: [2,3,4]
    println!("{hello} {world} {:?}", mid);
}

// Slices as parameters accept both String and &str
fn first_word(s: &str) -> &str {
    for (i, &b) in s.as_bytes().iter().enumerate() {
        if b == b' ' { return &s[..i]; }
    }
    s
}
```

**A slice** is a reference to a contiguous portion of a collection — it borrows, storing a pointer and a length, without owning the data.

### Viva — Slices

#### Q What is a slice?

**A** A borrowed reference to a contiguous range of a collection (string or array), holding a pointer and length without owning the elements.

#### Q Difference between String and &str?

**A** String is an owned, growable heap-allocated string; &str is a borrowed, fixed view (string slice). Accepting &str in functions is more flexible since it works with both.

## 8. Structs

```
struct User {
    username: String,
    email: String,
    active: bool,
}

// Tuple struct
struct Point(i32, i32);

impl User {
    // methods & associated fns
    fn new(username: String, email: String) -> User { // associated fn
        User { username, email, active: true } // field init shorthand
    }
    fn deactivate(&mut self) { // method (takes &mut self)
        self.active = false;
    }
    fn summary(&self) -> String { // method (takes &self)
        format!("{}", <{}>", self.username, self.email)
    }
}

fn main() {
    let mut u = User::new(String::from("abrar"), String::from("a@x.com"));
    println!("{}", u.summary());
    u.deactivate();
    let p = Point(3, 4);
    println!("{}", p.0, p.1);
}
```

### Viva — Structs

#### Q What are the kinds of structs in Rust?

**A** Classic structs with named fields, tuple structs with positional fields, and unit structs with no fields.

#### Q Method vs associated function?

**A** A method takes self (or &self/&mut self) and is called on an instance; an associated function (like a constructor `new`) doesn't take self and is called on the type with `::` syntax.

#### Q What is impl?

**A** An impl block defines methods and associated functions for a type, keeping behavior grouped with the data it operates on.

## 9. Enums & Pattern Matching

```
enum Shape {
    Circle(f64),           // variant with data
    Rectangle(f64, f64),   // variant with data
    Unit,                  // variant without data
}

fn area(s: &Shape) -> f64 {
    match s {              // match must be exhaustive
        Shape::Circle(r) => 3.14159 * r * r,
        Shape::Rectangle(w, h) => w * h,
        Shape::Unit => 0.0,
    }
}

fn main() {
    let shapes = [Shape::Circle(2.0), Shape::Rectangle(3.0, 4.0)];
    for sh in &shapes {
        println!("{:.2}", area(sh));
    }

    // if let: concise match for one pattern
    let maybe = Some(7);
    if let Some(n) = maybe {
        println!("got {n}");
    }
}
```

**match** is exhaustive: the compiler forces you to handle every variant (or add a `_` catch-all), eliminating whole classes of bugs.

### Viva — Enums & matching

#### Q What can Rust enums do that C enums can't?

**A** Rust enum variants can carry data of different types and amounts, making them algebraic data types — like a type-safe tagged union.

#### Q Why is match powerful?

**A** It's exhaustive — the compiler ensures every case is handled — and it deconstructs data out of variants in one step, preventing "forgot a case" bugs.

#### Q What is "if let"?

**A** Syntactic sugar for a match that only cares about one pattern, letting you handle a single case concisely without writing a full match.

## 10. Collections

---

### 10.1 Vector — growable array

```
let mut v: Vec<i32> = Vec::new();
v.push(1); v.push(2); v.push(3);
let first = v[0]; // index (panics if out of range)
if let Some(x) = v.get(10) { println!("{x}"); } // safe access -> None
for n in &v { println!("{n}"); }
let sum: i32 = v.iter().sum();
```

### 10.2 String — growable UTF-8 text

```
let mut s = String::from("foo");
s.push_str("bar");
s.push('!');
let combined = String::from("a") + "b"; // concatenation
```

### 10.3 HashMap — key-value store

```
use std::collections::HashMap;
let mut scores = HashMap::new();
scores.insert("blue", 10);
scores.insert("red", 50);
if let Some(v) = scores.get("blue") { println!("{v}"); }
*scores.entry("blue").or_insert(0) += 5; // update or insert default
for (k, val) in &scores { println!("{k}: {val}"); }
```

## Viva — Collections

#### Q Vec vs array?

**A** An array has a fixed compile-time length on the stack; a Vec is a heap-allocated, growable list whose size can change at runtime.

#### Q Why does HashMap.get return an Option?

**A** Because the key might be absent — Option forces you to handle the "missing" case explicitly rather than risk a null.

#### Q What does the entry API do?

**A** `entry(k).or_insert(default)` returns a mutable reference to the value, inserting a default if the key is missing — a clean update-or-insert pattern.

# 11. Error Handling

Rust has no exceptions. Recoverable errors use the `Result` enum; absence of a value uses `Option`; unrecoverable errors use `panic!`.

```
// Option<T> = Some(T) | None      -> a value that may be absent
// Result<T, E> = Ok(T) | Err(E)  -> an operation that may fail

use std::num::ParseIntError;

fn parse(s: &str) -> Result<i32, ParseIntError> {
    let n = s.parse::<i32>()?;    // ? propagates the error if Err
    Ok(n * 2)
}

fn main() {
    match parse("21") {
        Ok(v) => println!("ok {v}"),    // ok 42
        Err(e) => println!("err {e}"),
    }

    // Option handling
    let v = vec![1, 2, 3];
    match v.first() {
        Some(x) => println!("first {x}"),
        None    => println!("empty"),
    }

    // Convenience methods
    let n: i32 = "5".parse().unwrap_or(0); // default on error
    println!("{}", n);
}
```

**The ? operator** unwraps an `Ok` / `Some` value, or returns early with the `Err` / `None` from the enclosing function — making error propagation concise.

**unwrap() and expect()** extract the value but panic if it's `Err/None`. Fine for prototypes and tests; prefer proper handling or `?` in production code.

## Viva — Error handling

### Q How does Rust handle errors without exceptions?

**A** With return types: `Result<T,E>` for recoverable errors and `Option<T>` for possibly-absent values, forcing the caller to handle both cases. `panic!` is reserved for unrecoverable bugs.

### Q What does the ? operator do?

**A** On `Ok/Some` it yields the inner value; on `Err/None` it returns that early from the current function, propagating the error succinctly.



### **Q Difference between Option and Result?**

**A** Option expresses presence/absence (Some/None); Result expresses success/failure with an error value (Ok/Err).

### **Q When is panic! appropriate?**

**A** For unrecoverable situations or programming bugs (broken invariants) where continuing makes no sense; recoverable errors should use Result instead.

## 12. Generics

```
// Generic function: works for any type T that can be compared
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut biggest = &list[0];
    for item in list {
        if item > biggest { biggest = item; }
    }
    biggest
}

// Generic struct
struct Pair<T> { first: T, second: T }

impl<T: std::fmt::Display> Pair<T> {
    fn show(&self) { println!("{}", self.first, self.second); }
}

fn main() {
    println!("{}", largest(&[3, 7, 2, 9])); // 9
    println!("{}", largest(&['a', 'z', 'm'])); // z
    Pair { first: 1, second: 2 }.show();
}
```

**Monomorphization:** the compiler generates a concrete specialized version of generic code for each type used, so generics have zero runtime cost.

### Viva — Generics

#### Q What are generics for?

**A** Writing code that works over many types without duplication, using type parameters like `<T>`, often constrained by trait bounds.

#### Q Do generics slow the program down?

**A** No — through monomorphization the compiler produces specialized code per concrete type, giving the same speed as hand-written type-specific code.

#### Q What is a trait bound?

**A** A constraint like `T: PartialOrd` requiring the generic type to implement certain traits, so the function can use those capabilities.

## 13. Traits

A trait defines shared behavior — similar to an interface in other languages. Types implement traits to promise they provide that behavior.

```
trait Summary {
    fn summarize(&self) -> String;           // required method
    fn preview(&self) -> String {           // default method
        format!("(read more: {})", self.summarize())
    }
}

struct Article { title: String }
struct Tweet { user: String }

impl Summary for Article {
    fn summarize(&self) -> String { format!("Article: {}", self.title) }
}
impl Summary for Tweet {
    fn summarize(&self) -> String { format!("@{}", self.user) }
}

// Trait as a parameter (static dispatch)
fn notify(item: &impl Summary) {
    println!("Breaking! {}", item.summarize());
}

// Trait object (dynamic dispatch) for mixed collections
fn print_all(items: Vec<Box<dyn Summary>>) {
    for it in items { println!("{}", it.summarize()); }
}

fn main() {
    let a = Article { title: String::from("Rust 2.0") };
    notify(&a);
    println!("{}", a.preview());
}
```

### Viva — Traits

#### Q What is a trait?

**A** A collection of method signatures defining shared behavior; types implement it to guarantee they provide those methods — Rust's equivalent of an interface.

#### Q Static vs dynamic dispatch?

**A** Static dispatch (impl Trait / generics) resolves the concrete type at compile time with zero cost; dynamic dispatch (dyn Trait via a Box) resolves at runtime through a vtable, allowing heterogeneous collections.

**Q Can traits have default methods?**

**A** Yes — a trait can provide default implementations that implementors inherit and may override.

**Q What are derive traits?**

**A** `#[derive(Debug, Clone, PartialEq)]` auto-generates common trait implementations, saving boilerplate.

## 14. Lifetimes

*Lifetimes are how Rust ensures references never outlive the data they point to. Most are inferred; sometimes you annotate them explicitly.*

```
// The 'a annotation ties the output's lifetime to the inputs':  
// the returned reference is valid as long as BOTH inputs are.  
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() { x } else { y }  
}  
  
struct Excerpt<'a> {           // struct holding a reference needs a lifetime  
    part: &'a str,  
}  
  
fn main() {  
    let s1 = String::from("long string");  
    let s2 = String::from("short");  
    println!("{}", longest(&s1, &s2));  
  
    let novel = String::from("Call me Ishmael. Some years ago...");  
    let first = novel.split('.').next().unwrap();  
    let e = Excerpt { part: first };  
    println!("{}", e.part);  
}
```

**Lifetimes don't change how long values live** — they're annotations that describe relationships so the borrow checker can verify no reference dangles.

### Viva — Lifetimes

#### Q What is a lifetime?

**A** A compile-time label describing how long a reference is valid, letting the borrow checker guarantee references never outlive the data they point to.

#### Q Do lifetimes affect runtime?

**A** No — they're purely a compile-time construct for verification and add zero runtime cost or behavior change.

#### Q Why annotate a lifetime like 'a?

**A** When the compiler can't infer how input and output reference lifetimes relate, the annotation expresses that the returned reference lives as long as the referenced inputs.

## 15. Closures & Iterators

### 15.1 Closures — anonymous functions that capture their environment

```
fn main() {
    let factor = 3;
    let multiply = |x: i32| x * factor; // captures `factor` from scope
    println!("{}", multiply(5));      // 15

    let mut count = 0;
    let mut increment = || { count += 1; count }; // FnMut: mutates capture
    println!("{}", increment());              // 1
    println!("{}", increment());              // 2
}
```

**Closure traits:** `Fn` (borrows immutably), `FnMut` (borrows mutably), `FnOnce` (takes ownership, callable once) — the compiler picks the least restrictive that fits.

### 15.2 Iterators — lazy, composable sequences

```
fn main() {
    let nums = vec![1, 2, 3, 4, 5, 6];

    // Chained adapters are lazy until a consumer runs them
    let result: Vec<i32> = nums.iter()
        .filter(|&x| x % 2 == 0) // keep evens
        .map(|&x| x * x)         // square them
        .collect();              // consume into a Vec
    println!("{:?}", result);    // [4, 16, 36]

    let total: i32 = nums.iter().sum();
    let doubled: Vec<i32> = nums.iter().map(|x| x * 2).collect();
    let any_big = nums.iter().any(|&x| x > 5); // true
    println!("{total} {:?} {any_big}", doubled);
}
```

**Zero-cost iterators:** chained adapters compile to code as fast as a hand-written loop, while reading far more clearly.

### Viva — Closures & iterators

#### Q What is a closure?

**A** An anonymous function that can capture variables from its surrounding scope, stored in a variable and passed around like data.

**Q What does "lazy iterator" mean?**

**A** Iterator adapters like `map/filter` don't compute anything until a consuming method (`collect`, `sum`, `for`) drives them, so intermediate results aren't materialized.

**Q Difference between `iter`, `iter_mut`, and `into_iter`?**

**A** `iter` yields immutable references, `iter_mut` yields mutable references, and `into_iter` consumes the collection yielding owned values.

## 16. Smart Pointers

| Type                          | Purpose                                                        |
|-------------------------------|----------------------------------------------------------------|
| <code>Box&lt;T&gt;</code>     | Heap allocation; enables recursive types & owned trait objects |
| <code>Rc&lt;T&gt;</code>      | Reference counting for multiple owners (single-threaded)       |
| <code>Arc&lt;T&gt;</code>     | Atomic reference counting — thread-safe shared ownership       |
| <code>RefCell&lt;T&gt;</code> | Interior mutability; borrow rules checked at runtime           |

```
use std::rc::Rc;
use std::cell::RefCell;

fn main() {
    // Box: put data on the heap
    let boxed = Box::new(5);
    println!("{}", *boxed);

    // Rc: shared ownership, count tracks references
    let a = Rc::new(String::from("shared"));
    let b = Rc::clone(&a); // increments count, no deep copy
    println!("count = {}", Rc::strong_count(&a)); // 2

    // RefCell: mutate through an immutable reference (checked at runtime)
    let cell = RefCell::new(10);
    *cell.borrow_mut() += 5;
    println!("{}", cell.borrow()); // 15
}
```

### Viva — Smart pointers

#### Q What is Box used for?

**A** Allocating a value on the heap — needed for recursive types (like linked lists/trees) and for storing trait objects with a known pointer size.

#### Q Rc vs Arc?

**A** Both provide shared ownership via reference counting; Rc is for single-threaded use, while Arc uses atomic operations so it's safe to share across threads (at a small performance cost).

#### Q What is interior mutability?

**A** A pattern (via RefCell/Cell) that allows mutation through an immutable reference, moving borrow-rule enforcement from compile time to runtime.



## 17. Concurrency

Rust's ownership rules extend to threads, giving **fearless concurrency** — data races are caught at compile time.

```
use std::thread;
use std::sync::{Arc, Mutex, mpsc};

fn main() {
    // Spawning threads
    let handle = thread::spawn(|| {
        for i in 1..4 { println!("thread {i}"); }
    });
    handle.join().unwrap(); // wait for it to finish

    // Shared state with Arc<Mutex<T>>
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];
    for _ in 0..5 {
        let c = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            let mut n = c.lock().unwrap(); // lock for exclusive access
            *n += 1;
        }));
    }
    for h in handles { h.join().unwrap(); }
    println!("total = {}", *counter.lock().unwrap()); // 5

    // Message passing with channels
    let (tx, rx) = mpsc::channel();
    thread::spawn(move || { tx.send("hi").unwrap(); });
    println!("{}", rx.recv().unwrap()); // hi
}
```

**Mutex + Arc:** Arc shares ownership across threads; Mutex ensures only one thread accesses the data at a time. The compiler won't let you share mutable data unsafely.

### Viva — Concurrency

#### Q What is "fearless concurrency"?

**A** Rust's ownership and type system catch data races at compile time, so concurrent code that compiles is guaranteed free of that class of bug.

#### Q Why wrap shared data in Arc<Mutex<T>>?

**A** Arc gives multiple threads shared ownership; Mutex serializes access so only one thread mutates at a time — together they enable safe shared mutable state.

**Q What are the two main concurrency models Rust supports?**

**A** Shared-state concurrency (Mutex/Arc) and message passing via channels (mpsc), summarized as "don't communicate by sharing memory; share memory by communicating."

**Q What do Send and Sync mean?**

**A** Send marks types safe to transfer ownership across threads; Sync marks types safe to share by reference across threads. The compiler uses them to enforce thread safety.

## 18. Modules, Crates & Testing

```
// Organizing code with modules
mod math {
    pub fn add(a: i32, b: i32) -> i32 { a + b }    // pub = public
    pub mod advanced {
        pub fn square(x: i32) -> i32 { x * x }
    }
}

use math::advanced::square;

fn main() {
    println!("{}", math::add(2, 3));    // 5
    println!("{}", square(4));          // 16
}

// Built-in unit testing
#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn test_add() {
        assert_eq!(math::add(2, 2), 4);
    }
}
```

**Crate vs module:** a *crate* is a compilation unit (a library or binary); *modules* organize code within a crate. External crates are declared in Cargo.toml and fetched from crates.io.

### Viva — Modules & testing

#### Q Difference between a crate and a module?

**A** A crate is the unit of compilation (a binary or library); modules are namespaces that organize items inside a crate. Items are private by default and exposed with pub.

#### Q How do you write a test in Rust?

**A** Annotate a function with `#[test]` (usually inside a `#[cfg(test)]` module) and use `assert!/  
assert_eq!` macros; run them with `cargo test`.

## 19. Advanced Topics

---

### 19.1 Pattern matching depth

```
let pair = (0, -2);
match pair {
    (0, y) => println!("first zero, y={y}"),
    (x, 0) => println!("second zero, x={x}"),
    _      => println!("other"),
}

let n = 5;
match n {
    1..5 => println!("one to five"),    // range pattern
    _    => println!("something else"),
}
```

### 19.2 Operator overloading via traits

```
use std::ops::Add;
#[derive(Debug, Clone, Copy)]
struct V2 { x: f64, y: f64 }
impl Add for V2 {
    type Output = V2;
    fn add(self, o: V2) -> V2 { V2 { x: self.x+o.x, y: self.y+o.y } }
}
// V2{x:1.0,y:2.0} + V2{x:3.0,y:4.0} -> V2{x:4.0,y:6.0}
```

### 19.3 Associated types & the Iterator trait

```
struct Counter { count: u32 }
impl Iterator for Counter {
    type Item = u32;                // associated type
    fn next(&mut self) -> Option<u32> {
        if self.count < 5 { self.count += 1; Some(self.count) }
        else { None }
    }
}
// Now Counter works with all iterator adapters (map, filter, sum...)
```

### 19.4 Unsafe Rust

```
fn main() {
    let mut num = 5;
    let r1 = &num as *const i32;    // raw pointer
    unsafe {
        println!("{}", *r1);        // dereferencing needs unsafe
    }
}
```

**unsafe** unlocks five extra abilities (raw pointer deref, calling unsafe fns/FFI, mutable statics, etc.). It doesn't disable the borrow checker elsewhere — you're just promising the compiler you've upheld safety in that block. Keep unsafe blocks small and audited.

## Viva — Advanced

### Q What does unsafe actually allow?

**A** A small set of extra operations (dereferencing raw pointers, calling unsafe/FFI functions, accessing mutable statics, implementing unsafe traits). It doesn't turn off Rust's other safety checks; you vouch for correctness inside the block.

### Q What is an associated type?

**A** A type placeholder inside a trait (like `Iterator's Item`) that each implementor specifies, avoiding extra generic parameters and clarifying the API.

### Q How does operator overloading work?

**A** By implementing the relevant `std::ops` trait (`Add`, `Sub`, `Mul`...) for your type, which defines what the operator does for it.

### Q What is the "newtype" pattern?

**A** Wrapping a type in a single-field tuple struct to add type safety or implement external traits, with no runtime overhead.

**Mastery path:** the ownership → borrowing → lifetimes trio is the conceptual core of Rust. Once those click, traits and generics give you expressive power, and concurrency becomes safe by default. Build small projects with `cargo`, read the compiler's error messages carefully (they're famously helpful), and re-implement these examples from memory.